FeeFlow AI — Technical Design Document

Public GitHub repository: https://github.com/AnjiRangala/feeflow-ai

Deployed application: Not verified. No live application URL is available.

1. Project overview

FeeFlow AI is a fictional coaching-institute fee-management demonstration. It provides parent fee ledgers and mock checkout, accounts fee management and collection reporting, a mock reminder centre, pro-rata refund review, and a counsellor-facing student-support risk workflow.

2. Problem statement

Fee information, payment histories, due dates, and follow-up work need a consistent source of truth. The demonstration models a role-restricted ledger and review workflow without routing student/payment data to an external AI provider or payment/messaging gateway.

3. Proposed solution

The React client calls an authenticated Express API. MongoDB/Mongoose stores ledger and workflow records. Node performs financial calculations and authorization. A local FastAPI service provides an explainable rules-based risk flag; it is a supportive task prompt, not a decision engine.

4. Technology stack

Layer	Technologies
Web client	React, Vite, Tailwind CSS, React Router, Axios, Recharts
API	Node.js, Express, Zod, Helmet, CORS, express-rate-limit
Persistence	MongoDB Atlas or local replica set, Mongoose
Authentication	bcryptjs password hashes, HMAC-protected one-time codes, signed JWT
Student-support service	Python, FastAPI, Pydantic, Pandas
Synthetic-only experiment	scikit-learn, NumPy; not used by prediction endpoint
Tests	Node built-in test runner, pytest, frontend production build

5. Architecture diagram

■■■■■■■■■■■■■■■■■■■■■ React / Vite browser ■■■■■■■■■■■■■■■■■■■■■

■ Parent portal ■ Accounts dashboard ■ Counsellor risk monitor ■

■■

■ JSON + bearer JWT

▼

■■■■■■■■■■■■■■■■■■■■■■■ Express API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■ Auth/OTP ■ Zod validation ■ role middleware ■ services ■

```
■ Fee ledger ■ mock payments ■ reminders ■ refunds ■ risk tasks ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Mongoose ■ local HTTP only
▼ ▼
■■■■■■■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ MongoDB ■ ■ FastAPI rules baseline ■
■ users / students ■ ■ /health ■
■ fee plans / payments ■ ■ /predict-risk ■
■ reminders / refunds ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ OTP / counsellor ■
■■■■■■■■■■■■■■■■■■■■■■■■
```

Synthetic scikit-learn comparison is an offline experiment and does not
participate in predictions or write to MongoDB.

6. Database schema

All monetary fields are safe integer paise. Student/business relationships use the unique human-readable `studentId`; payment and reminder records reference embedded MongoDB installment IDs.

Collection/model	Purpose
User	Normalized unique email, selected-out bcrypt hash, role, optional linked student, language preference, active state
Student	Fictional/profile data, parent contact, attendance, test trend history, payment promises
FeePlan	Unique student plan, gross/discount/net amounts, 3–4 embedded installments and payment references
Payment	Completed or failed transaction history, unique transaction ID, amount and installment
MockPaymentOrder	Short-lived parent-bound order with server-computed amount and settlement state
Reminder	Idempotent scheduled message, mock delivery/retry state, failure/cancellation audit reason
Refund	Versioned calculation inputs/breakdown and pending/approved/rejected review state
CounsellorTask	Risk feature snapshot, score/explanation, status and one-time counsellor feedback
OtpChallenge	HMAC code digest, expiry and attempt count; no plaintext OTP

The demo seed uses stable keys and insert-only upserts. It refuses to modify colliding unmarked records and verifies seed-managed identities/passwords rather than silently replacing them.

7. API documentation

All API routes are under `/api`. Amounts are integer paise. Except login and OTP verification, routes require a bearer JWT and enforce roles in the API.

Method and path	Access	Behavior
`GET /health`	Public	API and MongoDB readiness
`POST /auth/login`	Public, rate-limited	Verify password and issue OTP challenge
`POST /auth/verify-otp`	Public, rate-limited	Consume one-time code and return JWT/user
`GET /auth/me`	Authenticated	Current active user profile

`GET /fees/students`	Parent, accounts	Parent's linked ledger(s) or accounts list
`GET /fees/student/:studentId`	Parent, accounts	Student ledger with parent IDOR guard
`POST /fees/create-plan`	Accounts	Create a validated 3/4-installment plan
`GET /fees/summary`	Accounts	Ledger totals
`GET /fees/dashboard`	Accounts	Collections, trends, group metrics, ageing
`GET /fees/export.csv`	Accounts	Formula-safe CSV export
`POST /payments/record`	Accounts	Record validated payment transaction
`POST /parent-payments/orders`	Parent	Create server-priced linked-student order
`POST /parent-payments/orders/:orderId/complete`	Parent	Run development mock outcome; backend validates/signs settlement
`GET /parent-payments/receipts/:paymentId`	Parent, accounts	Authorized completed-payment receipt
`GET/POST /reminders/preferences`	Parent	Read/update English or Hindi preference
`GET /reminders`	Accounts	Filterable reminder centre
`POST /reminders/process`	Accounts	Process a mock reminder cycle
`POST /reminders/:reminderId/retry`	Accounts	Retry an eligible failed reminder
`GET /refunds`	Accounts	List calculation/review records
`POST /refunds/calculate`	Accounts	Create a server-calculated report
`POST /refunds/:refundId/approve`	Accounts	Approve a pending report; no disbursement
`POST /refunds/:refundId/reject`	Accounts	Reject a pending report
`GET /risks/tasks`	Accounts, counsellor	Read risk follow-up tasks
`POST /risks/assess`	Accounts	Derive features server-side and request local assessment
`POST /risks/tasks/:taskId/feedback`	Counsellor	Record one valid/false-flag review
`POST /assistant/ask`	Authenticated accounts workflow	Ask deterministic ledger-backed fee question
`GET /health`	FastAPI local service	Report service and prediction method
`POST /predict-risk`	FastAPI local service	Return validated explainable rules assessment

Payment and migration operations require transaction-capable MongoDB (replica set or mongos). The API fails explicitly if transactions are unavailable; support has not been verified against the currently configured Atlas deployment.

8. Authentication and roles

Passwords are hashed with bcrypt. Login validates credentials, stores only an HMAC of a random six-digit OTP, expires and limits its attempts, then consumes the challenge before issuing a short-lived JWT. JWT identity is reloaded from the database, and authorization uses the current persisted role.

Role	Scope
Parent	Only the active linked student, ledger, language preference, own mock orders and receipts
Accounts	Fee records, payments, dashboards, reminders, refunds, and requesting risk assessments
Counsellor	Read risk tasks and record valid/false-flag feedback; no fee-management access

9. Fee calculation logic

The ledger uses integer paise. Net fee equals gross fee minus discount. A plan must contain three or four installments whose amounts sum to net fee. Equal splits distribute remainder paise deterministically. Each installment's outstanding balance is amount minus paid amount, bounded by validation against overpayment. An unpaid balance past its UTC due date is overdue; overdue days are whole UTC days. Student totals are derived from installment data rather than trusted client totals.

10. Payment workflow

For parent checkout, the backend resolves the parent-linked student and installment, calculates the outstanding amount, creates a short-lived unique order and signs the simulated result server-side. Settlement rechecks the order, expiry, signature, and current balance. Payment history, installment balance/status, order state, and fully-paid reminder cancellation are written transactionally. Duplicate transaction IDs/replays and overpayments are rejected. The mock simulator is disabled in production; this is not a payment-provider integration.

Accounts payment recording uses the same ledger transaction discipline. Receipts are generated from persisted completed payment data and are access-controlled. Mongo transaction behavior has not been tested against the current configured deployment.

11. Reminder automation

Default schedule offsets are 3 days before, on the due date, and 3, 7, and 15 days after. Offsets, polling, catch-up window and retry delays are configurable. Idempotency keys prevent duplicate schedules. The worker checks current payment state just before delivery, refreshes partial balances, stops reminders for fully paid installments and stores audit/failure/cancellation metadata. A mock provider only persists delivery state; it sends no SMS.

12. Refund calculation

The disclosed demonstration policy is:

`Refundable = max(0, amount paid – fee earned – non-refundable charges)`

`Fee earned = (net course fee / course duration in days) × completed course days`

Inputs and policy version are captured; dates use whole UTC-day differences and paise rounding. Accounts users review/approve/reject; approval never automatically issues money. This is a sample assumption, not a legal formula or an institutional policy.

13. AI model architecture

The API derives missed installments, overdue days, attendance, latest test-score delta and broken payment promises. Local FastAPI validates ranges and returns a deterministic explainable rules score, feature contributions, reasons and recommended supportive action. The service labels its method `explainable_rules_baseline`.

There is no verified labelled historical dropout dataset. An offline scikit-learn StandardScaler/LogisticRegression comparison uses synthetic features and synthetic labels derived from the rules threshold with 8% label noise. It uses a fixed-seed stratified 75/25 split and fits preprocessing on training data only. Its metrics measure synthetic-label recovery only; the ML pipeline is not used by the API.

14. Human review safeguards

Risk output is a task prompt, not a probability or decision. Medium/high tasks require counsellor review; feedback is recorded with reviewer and time. A risk assessment cannot change fees, payments, reminders, enrollment or penalties. Accounts staff cannot submit counsellor feedback. Risk scores can be wrong or biased by stale/missing data; no automated adverse action is permitted.

15. UI/UX decisions

The React app uses role-specific navigation, responsive cards/tables, readable status labels, loading/error/empty states, and mobile-first parent payment actions. Accounts users see the dashboard, searchable ledger, reminder centre, refund calculator and risk monitor. Counsellors see risk tasks and review controls only. Recharts visualizes collection trends and ageing/group summaries.

The login screen was rendered from the production build at 390×844, 768×1024, and 1440×1000. This exposed and led to a fix for a tablet-width intro-panel overflow; the screenshots are in `docs/screenshots/`. Authenticated dashboard screens and payment/refund workflows were not browser-tested.

16. Testing strategy

- Backend: Node test runner covers authentication/OTP, financial calculations, ledger APIs, mock payments, receipts, reminders, refunds, role checks and risk API behavior.
- AI service: pytest covers health, missing/invalid features, low/high rules assessments and reproducible synthetic comparison.
- Frontend: Vite production build.
- Optional database tests: seed/login/isolation against configured MongoDB, and an isolated-database test that creates/removes a uniquely named QA database.

Current verification: backend 85 tests, 83 passed, 0 failed, 2 documented skips; AI service 7 passed; frontend production build passed; Playwright 6 passed; focused payment/refund/provider tests 6 passed; JavaScript/Python syntax checks and `git diff --check` passed. The local browser tests use synthetic API fixtures, and external provider, SMTP, deployment, and production database acceptance remain unverified.

17. Security measures

The API uses Helmet, a configured single-origin CORS policy, JSON body-size limit, strict Zod schemas, parameterized Mongoose filters, bcrypt password hashes, OTP HMAC/expiry/attempt limits, JWT validation, and role middleware. Authentication endpoints use a 10-request/15-minute limiter. SMTP requires STARTTLS unless implicit TLS is configured. `.env` files and build/dependency artifacts are ignored; only variable names appear in example files.

The public repository preserves the existing local Git history and was published without force-pushing. Repository publication checks found no tracked environment files, private keys, or live credential patterns. Configure HTTPS, least-privilege database access, SMTP and production secrets at deployment.

18. Assumptions

- Demo identities and student records are fictional.
- Monetary amounts in APIs/database are integer paise.
- Refund formula and GST receipt note are explicitly labelled project assumptions.
- Mock gateway and messaging provider do not contact external providers.
- Risk scores are deterministic rules and not dropout probabilities.
- Production deployment must supply transaction-capable MongoDB, SMTP and secure environment configuration.

19. Assignment compliance audit

Assignment requirement	Status	Evidence or limitation
Fee ledger and integer-paise balances	Implemented and verified	Server-derived plans, installment balances, payments, receipts, and ledger tests
Three/four-installment rules	Implemented and verified	Validated plan creation and deterministic remainder handling
Parent OTP login	Implemented but not externally verified	OTP expiry, HMAC, attempts, and atomic consumption are tested; SMTP delivery is not
Parent dues and payment links	Implemented and verified locally	Parent-owned mock order flow and receipt tests
Reminder automation and cancellation after payment	Implemented and verified locally	Idempotent schedules, mock delivery state, and transactional cancellation
Pro-rata refund calculator	Implemented and verified locally	Ledger-backed policy snapshot and validation tests
Centre/batch Accounts dashboard	Implemented and verified locally	Aggregates, trends, ageing, CSV export, and browser dashboard tests
AI dropout-risk flag	Implemented and verified locally	Explainable rules baseline creates counsellor tasks only
False-positive handling	Implemented and verified locally	Counsellor `valid`/`false_flag` feedback is persisted
Hindi/English support	Partially implemented	Reminder language preference exists; full localization is not present
GST receipts	Partially implemented	Receipts include configurable GST note; tax calculation/registration is not implemented
DPDP parental consent	Not implemented	No consent record or consent workflow exists
Tally export	Not implemented	Accounts CSV export exists; Tally-specific export does not
Attendance/test-score upload	Partially implemented	Fields are consumed by risk logic; upload workflow is not present
Unmatched payment Accounts queue	Not implemented	No unmatched-payment queue or reconciliation UI exists

The core build was prioritized around a reliable ledger, parent visibility, payment/reminder lifecycle, refund transparency, Accounts operations, and human-reviewed risk support. The excluded items require additional product, legal, provider, or data-governance decisions and were not represented as implemented.

20. Limitations

- No verified public GitHub URL, deployment URL or accessible hosted application was found.
- Current MongoDB authentication failed; seed idempotency, real persistence, Atlas permissions and transaction support remain unverified.
- OTP login requires configured delivery outside development.
- Mock payment is not a real gateway; mock reminders are not SMS.
- No historical labelled risk dataset or real predictive validation exists.
- Authenticated browser E2E, production security testing and deployment were not completed.

21. Future improvements

Connect a verified project remote and review its history; deploy to a selected hosting provider; add a transaction-capable test database and CI integration environment; integrate a real payment provider using signed callbacks and reconciliation; adopt institution-approved refund policy; add real messaging only with consent and delivery governance; conduct accessibility and viewport testing; and evaluate risk support with representative, consented, audited historical data before any predictive use.